



AI-POWERED TEST CASE GENERATOR

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF
THE DEGREE OF

B.TECH. (INFORMATION TECHNOLOGY)

TO

RK UNIVERSITY, RAJKOT

SUBMITTED BY

Name of Student

Rutvik Shiyal

Enrollment No.

24SOEIT13019

Under the Guidance of

Internal Guide

Prof. Jay Pithadiya

Assistant Professor

RK University, Rajkot

December 2026



SCHOOL OF ENGINEERING, RK UNIVERSITY, RAJKOT

CANDIDATE'S DECLARATION

I hereby certify that I am the sole author of this project work and that neither any part of this project work nor the whole of the project work has been submitted for a degree to any other University or Institution. I certify that, to the best of my knowledge, my project work does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in my project document, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. I declare that this is a true copy of my project work, including any final revisions, as approved by my project review committee.

Signature of Student (S): _____
Rutvik Shiyal (24SOEIT13019)

Date: _____

Place: _____

CERTIFICATE

This is to certify that the work which is being presented in the Project Report entitled “AI-POWERED TEST CASE GENERATOR”, in partial fulfillment of the requirement for the award of the degree of B.Tech. (Information Technology Engineering) submitted to the School of Engineering, RK University, is an authentic record of my own work carried out during a period from April 2026 to December 2026.

The matter presented in this Project Report has not been submitted by me for the award of any other degree elsewhere.

Signature of Student (S):

Rutvik Shiyal (24SOEIT13019)

This is to certify that the above statement made by the student is correct to the best of my knowledge.

Internal Guide

Prof. Jay Pithadiya
Assistant Professor,
RK University,
Rajkot

Head of Department

Dr. Chetan Shingadiya
CE / IT,
School of Engineering,
RK University, Rajkot



SCHOOL OF ENGINEERING, RK UNIVERSITY, RAJKOT

ACKNOWLEDGEMENT

It is with deep gratitude and respect that I express my sincere appreciation to everyone who has contributed to the successful completion of this project. Their support, guidance, and encouragement have been invaluable throughout this journey.

I would like to extend my deepest gratitude to my internal guide, Prof. Jay Pithadiya, Assistant Professor, School of Engineering, RK University, for the constant guidance, constructive feedback, and intellectual support provided during the research and development phases of this application. The insights and suggestions on component layout and logical workflows greatly improved the software design.

I am also highly indebted to Dr. Chetan Shingadiya, Head of Department (CE/IT), School of Engineering, for providing conducive academic conditions and state-of-the-art facilities to pursue practical software engineering.

Finally, I would like to express my appreciation to my peers and family, whose patience, understanding, and encouragement enabled me to balance academic timelines and software deliverables. This project is a result of continuous learning and collective guidance, standing on the shoulders of the mentors who paved the way.

Rutvik Shiyal

Enrollment No: 24SOEIT13019

ABSTRACT

Software testing is an essential phase of the software development life cycle, yet authoring test cases manually from requirement documents is repetitive, time-consuming, and prone to missed scenarios. Testers often have to derive positive, negative, and boundary scenarios by hand for every user story, feature description, or API endpoint, which slows agile sprints and leads to inconsistent test documentation.

To address these challenges, this project presents the AI-Powered Test Case Generator, a full-stack web application that converts textual software requirements into structured test suites. The system is developed using a React 18 single-page frontend styled with Tailwind CSS and a Node.js and Express.js backend that exposes REST APIs. Test cases are generated through the Google Gemini API, with a context-aware NLP rule engine acting as an automatic fallback so that generation continues even when the cloud service is unavailable or rate limited.

Key features of the application include support for three specification formats (User Story, Feature Description, and API Endpoint Spec), one-click preset templates, an interactive test case table with search, filtering, full CRUD operations, and live execution status tracking, and multi-format export to Excel, PDF, CSV, and clipboard. The platform also provides role-based access control across Guest, Registered Engineer, and Administrator tiers, a freemium generation quota, a history archive of generated suites, and an Admin Portal for user management and server telemetry.

By automating the derivation of positive, negative, and boundary scenarios from a requirement, the AI-Powered Test Case Generator reduces manual effort, improves the consistency of test documentation, and provides a practical foundation for AI-assisted quality assurance.

ABBREVIATIONS AND NOTATIONS

The following table lists the definitions of abbreviations, terminology, and naming conventions used throughout this project report:

Abbreviation / Term	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
CTA	Call To Action
DB	Database
DOM	Document Object Model
DoS	Denial of Service
DTO	Data Transfer Object
FK	Foreign Key
GIS	Google Identity Services
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
OAuth	Open Authorization
PDF	Portable Document Format
PK	Primary Key
QA	Quality Assurance
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single-Page Application
UI	User Interface
UX	User Experience
XSS	Cross-Site Scripting
1:1	One-to-One Relationship
1:N	One-to-Many Relationship

List of Figures

Figure No. / Title	Page No.
Figure 5.1: Database Relationship Diagram	2
Figure 5.2: AI-Powered Test Case Generator System Flow	5
Figure 5.3: Dashboard Overview Interface	7
Figure 5.4: Generator Workspace Interface	7
Figure 5.5: Interactive Test Case Table Interface	8
Figure 5.6: Add / Edit Test Case Modal	8
Figure 5.7: Multi-Format Export Toolbar	9
Figure 5.8: Admin Portal Interface	9
Figure 5.9: AI-Powered Test Case Generator System Architecture	10
Figure 8.1: Dashboard & QA Project Telemetry	19
Figure 8.2: Test Case Generator Workspace with Preset Templates	20
Figure 8.3: AI Test Case Generation in Progress	20
Figure 8.4: Interactive Test Cases Data Table with Live Execution Tracker	21
Figure 8.5: Add Custom Test Case Modal Screen	22
Figure 8.6: Inline Edit Test Case Modal Screen	22
Figure 8.7: Multi-Format Real Export Subsystem Toolbar	23
Figure 8.8: Test Suite History Archive Screen	24
Figure 8.9: Authentication & Google Sign-In Modal Screen	24
Figure 8.10: Pricing & Subscription Upgrade Modal Screen	25
Figure 8.11: Admin Portal: Multi-Tenant User Management Screen	26
Figure 8.12: Admin Portal: AI Telemetry & Gemini Health Screen	26
Figure 8.13: Admin Portal: System Permissions & Auto-Allow Screen	27

List of Tables

Table No. / Title	Page No.
Table 5.1: Object/Class to Table Mapping	1
Table 5.2: Database Relationship Summary	2
Table 5.3: Test Suite Entity Schema (`test_suites`)	3
Table 5.4: Test Case Item Schema (`test_cases`)	3
Table 5.5: User Account & Quota Schema (`users`)	4
Table 6.1: Implementation Environment	11
Table 7.1: Testing Methods and Verification Scope	16
Table 7.2: Comprehensive Functional Test Cases (TC-01 to TC-20)	17
Table 7.3: Security Test Cases (SEC-01 to SEC-06)	18

TABLE OF CONTENTS

Chapter / Title	Page No.
CANDIDATE'S DECLARATION	I
CERTIFICATE	II
ACKNOWLEDGEMENT	III
ABSTRACT	IV
ABBREVIATIONS AND NOTATIONS	V
List of Figures	VI
List of Tables	VII
TABLE OF CONTENTS	VIII
CHAPTER 5: SYSTEM DESIGN	1
5.1 Database Design / Data Structure Design	1
5.1.1 Mapping objects/classes to tables	1
5.1.2 Tables and Relationship	1
5.1.3 Logical Description of Data	4
5.2 System Procedural Design	5
5.2.1 Flow chart or activity design	5
5.3 Input/Output and Interface Design	6
5.3.1 Samples of Forms and Interface	6
5.3.2 Access Control and Security	9
5.4 System Architecture Design	10
CHAPTER 6: IMPLEMENTATION PLANNING AND DETAILS	11
6.1 Implementation Environment	11
6.2 Program / Modules Specification	11
6.3 Security Features	13
6.4 Coding Standards	13
6.5 Sample Coding	13
CHAPTER 7: TESTING	16
7.1 Testing Plan	16
7.2 Testing Strategy	16
7.3 Testing Methods	16
7.4 Test Cases	17

Chapter / Title	Page No.
CHAPTER 8: SCREENSHOTS AND USER MANUAL	19
8.1 Application Screenshots	19
8.2 User Manual	27
CHAPTER 9: LIMITATIONS AND FUTURE ENHANCEMENT	30
9.1 Limitations	30
9.2 Future Enhancement	30
CHAPTER 10: CONCLUSION AND DISCUSSION	31
10.1 Conclusion	31
10.2 Discussion	31
10.3 References and Bibliography	31

CHAPTER 5: SYSTEM DESIGN

5.1 DATABASE DESIGN / DATA STRUCTURE DESIGN

The AI-Powered Test Case Generator adopts a high-efficiency decoupled architecture combining client-side persistent state (HTML5 LocalStorage) and server-side memory structures, optimized for high throughput and rapid test scenario generation. In modern software quality engineering, test case suites are composed of hierarchical, context-sensitive entities comprising precondition steps, execution test vectors, expected outcomes, priority weights, and runtime execution statuses.

Unlike monolithic relational databases that introduce I/O latency during quick agile sprint testing, the application encapsulates every test suite as an independent document entity with unique suite IDs, timestamp boundaries, and contextual categorization tags.

5.1.1 MAPPING OBJECTS/CLASSES TO TABLES

In the JavaScript and Node.js execution environment, domain entities are implemented using structured object schemas. To guarantee strict schema consistency across the REST API interface and client storage, JSON document schemas are mapped directly to relational-style entities. Each primary object—comprising Test Suites, Test Cases, User Accounts, and System Telemetry—is normalized to prevent data redundancy while ensuring seamless serialization for Excel, PDF, and CSV exports.

The mapping between JavaScript Object Models, REST API DTO payloads, and persistent storage structures is summarized below:

Table 5.1: Object/Class to Table Mapping

Application Class / Entity	Data Store / Payload Type	Purpose / Functional Scope
TestSuite	JSON Document / LocalStorage	Aggregates generated test scenarios under a unified requirement context.
TestCaseItem	Normalized Array of Objects	Represents individual positive, negative, and boundary test scenarios.
UserAccount	Client Token & Server Memory	Manages user credentials, authentication provider, plan limits, and roles.
TelemetryAudit	In-Memory Server Metrics	Captures server uptime, Gemini API latency, request count, and heap usage.
ExportPayload	SheetJS / jsPDF Buffer	Temporary in-memory structure for compiling .xlsx, .pdf, and .csv files.

5.1.2 TABLES AND RELATIONSHIP

The relational data structures governing the AI-Powered Test Case Generator platform consist of four core entities: User Account, Test Suite, Test Case Item, and Telemetry Audit Log.

- A 1:N (One-to-Many) relationship exists between UserAccount and TestSuite.

- A **1:N (One-to-Many)** relationship exists between each `TestSuite` and its constituent `TestCaseItem` records.
- A **1:1 (One-to-One)** dependency links execution metrics to the active browser workspace session.

Table 5.2: Database Relationship Summary

Relationship	Type	Description
UserAccount → TestSuite	One-to-Many (1:N)	One user account can own multiple test suites
TestSuite → TestCaseItem	One-to-Many (1:N)	One test suite contains multiple test cases
TelemetryAudit → Workspace Session	One-to-One (1:1)	Execution metrics are linked to the active browser workspace session

Figure 5.1: Database Relationship Diagram

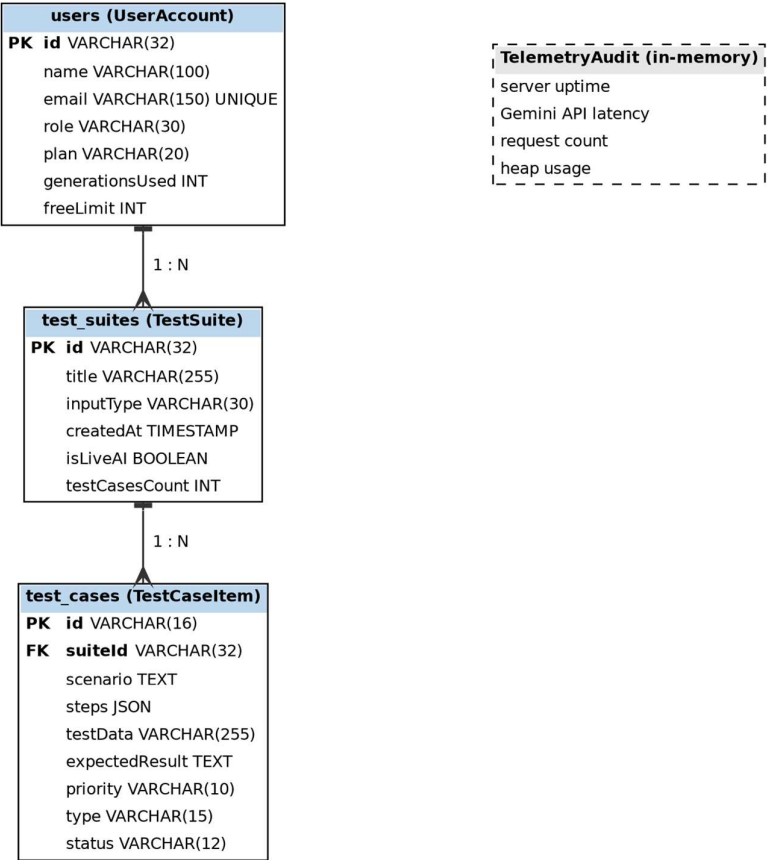


Table 5.3: Test Suite Entity Schema (`test_suites`)

Field Name	Data Type	Constraint	Nullability	Description
<code>id</code>	String (VARCHAR 32)	PRIMARY KEY	NOT NULL	Unique suite identifier (e.g., SUITE-1042, SUITE-8821)
<code>title</code>	String (VARCHAR 255)	None	NOT NULL	Requirement headline, feature description, or API summary
<code>inputType</code>	Enum (VARCHAR 30)	CHECK IN ('User Story', 'Feature Description', 'API Endpoint Spec')	NOT NULL	Specification classification format
<code>createdAt</code>	Timestamp (ISO 8601)	None	NOT NULL	Exact date and time of test suite generation
<code>isLiveAI</code>	Boolean	DEFAULT true	NOT NULL	Flag indicating whether generated via Gemini AI (<code>true</code>) or NLP fallback (<code>false</code>)
<code>testCasesCount</code>	Integer	CHECK (<code>testCasesCount >= 0</code>)	NOT NULL	Total count of child test cases contained in the suite

Table 5.4: Test Case Item Schema (`test_cases`)

Field Name	Data Type	Constraint	Nullability	Description
<code>id</code>	String (VARCHAR 16)	PRIMARY KEY	NOT NULL	Standardized case ID (e.g., TC-US-001, TC-FD-002, TC-API-003)
<code>suiteId</code>	String (VARCHAR 32)	FOREIGN KEY	NOT NULL	References parent <code>test_suites.id</code>
<code>scenario</code>	Text (TEXT)	None	NOT NULL	Specific test scenario objective or verification statement
<code>steps</code>	Array of Strings (JSON)	JSON Array	NOT NULL	Ordered, numbered sequence of execution steps
<code>testData</code>	Text (VARCHAR 255)	None	NOT NULL	Sample input parameters, payloads, or authentication headers
<code>expectedResult</code>	Text (TEXT)	None	NOT NULL	Expected system behavior, UI confirmation, or HTTP response
<code>priority</code>	Enum (VARCHAR 10)	CHECK IN ('High', 'Medium', 'Low')	NOT NULL	Risk-weighted execution priority ranking
<code>type</code>	Enum (VARCHAR 15)	CHECK IN ('Positive', 'Negative', 'Boundary')	NOT NULL	Quality verification category

Field Name	Data Type	Constraint	Nullability	Description
status	Enum (VARCHAR 12)	CHECK IN ('Pending', 'Passed', 'Failed', 'Blocked')	NOT NULL	Real-time interactive execution status

Table 5.5: User Account & Quota Schema (users)

Field Name	Data Type	Constraint	Nullability	Description
id	String (VARCHAR 32)	PRIMARY KEY	NOT NULL	Unique user identifier (e.g., USR-ADMIN-01, USR-GUEST)
name	String (VARCHAR 100)	None	NOT NULL	Full name of registered engineer or administrator
email	String (VARCHAR 150)	UNIQUE	NOT NULL	Registered email address
role	Enum (VARCHAR 30)	CHECK IN ('Administrator', 'QA Automation Lead', 'Developer')	NOT NULL	Role-Based Access Control tier
plan	Enum (VARCHAR 20)	CHECK IN ('Free Tier', 'Pro', 'Enterprise')	NOT NULL	Subscription quota plan
generationsUsed	Integer	DEFAULT 0	NOT NULL	Number of AI test suite generations consumed
freeLimit	Integer	DEFAULT 10	NOT NULL	Generation limit before pricing modal intercept (999999 for Pro)

5.1.3 LOGICAL DESCRIPTION OF DATA

The system enforces strict domain integrity rules across all data attributes:

1. **Requirement Text Boundary:** Specification inputs are capped between 10 and 1,000 characters to prevent prompt token overflow and maintain sub-second LLM inference latency.
2. **Priority Hierarchy:**
 - **High:** Applied to authentication, authorization, financial calculations, and irreversible data mutations.
 - **Medium:** Applied to standard CRUD operations, navigation, and state queries.
 - **Low:** Applied to aesthetic formatting, tooltips, and optional sorting filters.
3. **Scenario Distribution Matrix:** Every generated test suite is automatically balanced across three testing dimensions:
 - **Positive Scenarios (Happy Path):** 40% – 50%
 - **Negative Scenarios (Error Handling & Invalid Data):** 30% – 40%

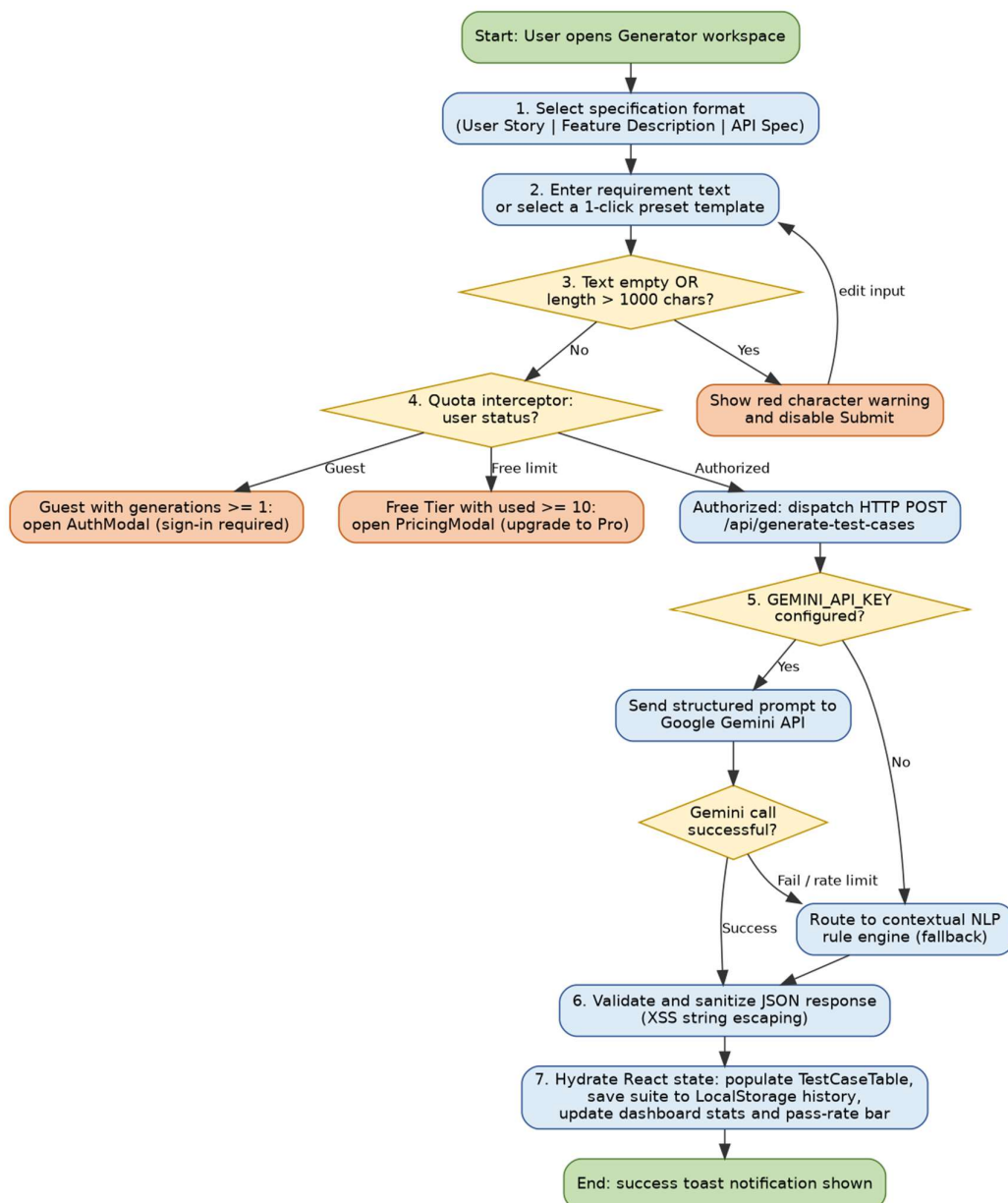
- Boundary Scenarios (Limit thresholds & capacity): 15% – 20%
4. **Execution Status Lifecycle:** Test statuses transition predictably: Pending → Passed → Failed → Blocked → Pending.

5.2 SYSTEM PROCEDURAL DESIGN

5.2.1 FLOW CHART OR ACTIVITY DESIGN

The procedural execution flow of the AI-Powered Test Case Generator orchestrates input ingestion, character validation, freemium quota gating, dual-engine AI dispatch, response parsing, and interactive DOM hydration.

Figure 5.2: AI-Powered Test Case Generator System Flow



5.3 INPUT/OUTPUT AND INTERFACE DESIGN

5.3.1 SAMPLES OF FORMS AND INTERFACE

The graphical user interface is structured into specialized functional components designed with modern visual aesthetics (glassmorphic navigation, high-contrast dark mode, and responsive layout):

1. **Top Navigation Bar (`Navbar.jsx`):** Features brand logo, active tab navigation pills (Dashboard, Generator, History, Admin Portal), quota usage counter badge, dark mode toggle button, and profile trigger.
2. **Dashboard Overview (`DashboardPage.jsx`):** Renders executive QA metrics (Total Test Suites, Active Test Cases, Total Scenarios Generated), interactive Pass Rate progress indicator, and Recent Activity feed.
3. **Generator Workspace (`GeneratorPage.jsx`):** Form interface with format selector dropdown, 1-click Preset Template pills (UTMS Student Bus QR Attendance, E-Commerce Checkout, POST Auth API), live character counter (1000 max), and animated loading status.
4. **Interactive Data Table (`TestCaseTable.jsx`):** Advanced tabular grid equipped with real-time keyword search, Priority and Type dropdown filters, expandable execution step rows, inline edit modals, and clickable status badges (Passed, Failed, Blocked, Pending).
5. **Add / Edit Custom Test Case Modal:** Modal dialog with structured inputs for Test ID, Scenario Title, Step-by-Step Instructions, Test Data, Expected Outcome, Priority, and Scenario Type.
6. **Multi-Format Export Toolbar (`ExportToolbar.jsx`):** Segmented scope controller (All vs Selected Rows) with one-click triggers for native Excel (`.xlsx`), landscape PDF (`.pdf`), CSV (`.csv`), and formatted plain text clipboard copying.
7. **Admin Portal (`AdminPortalPage.jsx`):** Super Admin console featuring multi-tenant user management table, 1-click Pro upgrades, quota resets, and real-time Google Cloud API telemetry.

Sample screens of the major interfaces described above are shown below.

Figure 5.3: Dashboard Overview Interface

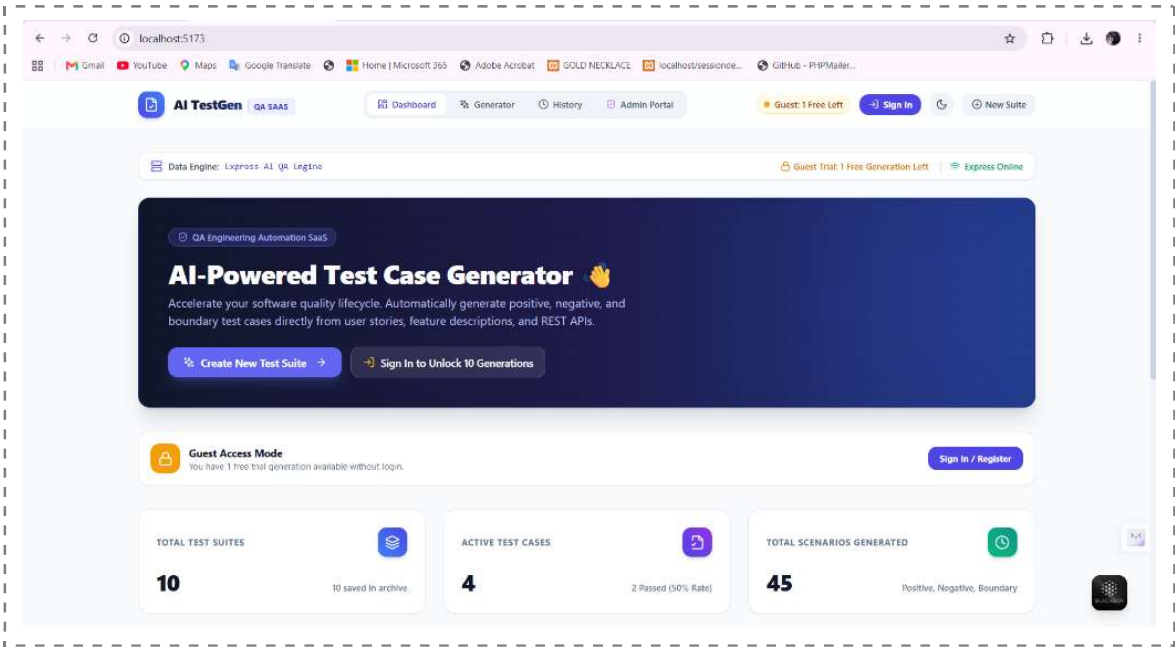


Figure 5.4: Generator Workspace Interface

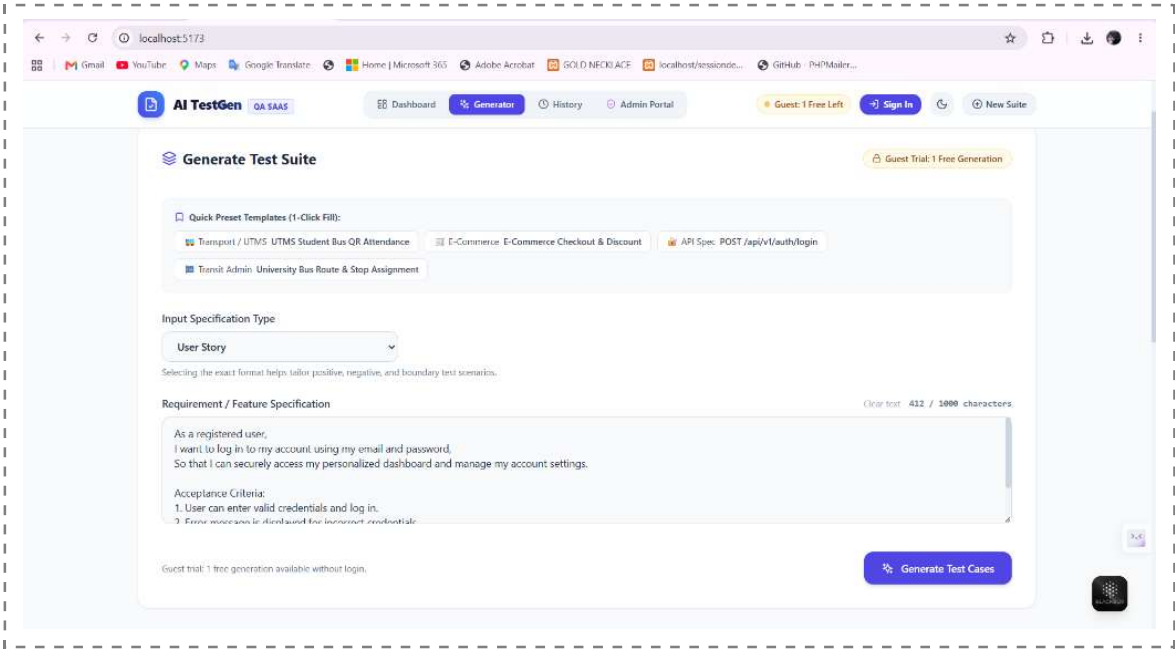


Figure 5.5: Interactive Test Case Table Interface

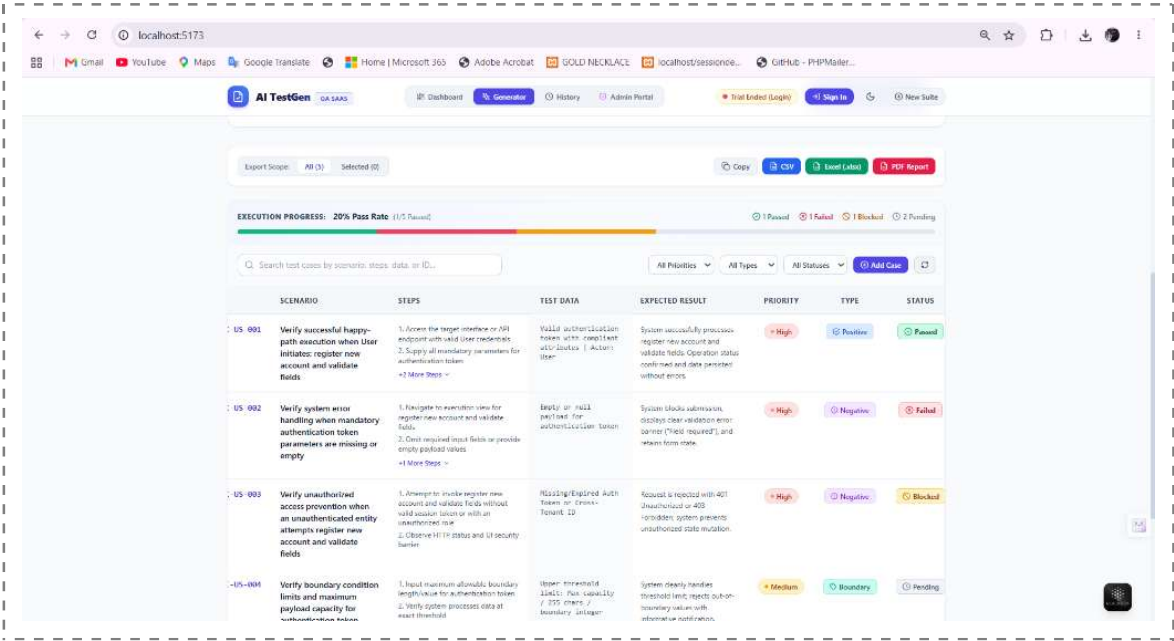


Figure 5.6: Add / Edit Test Case Modal

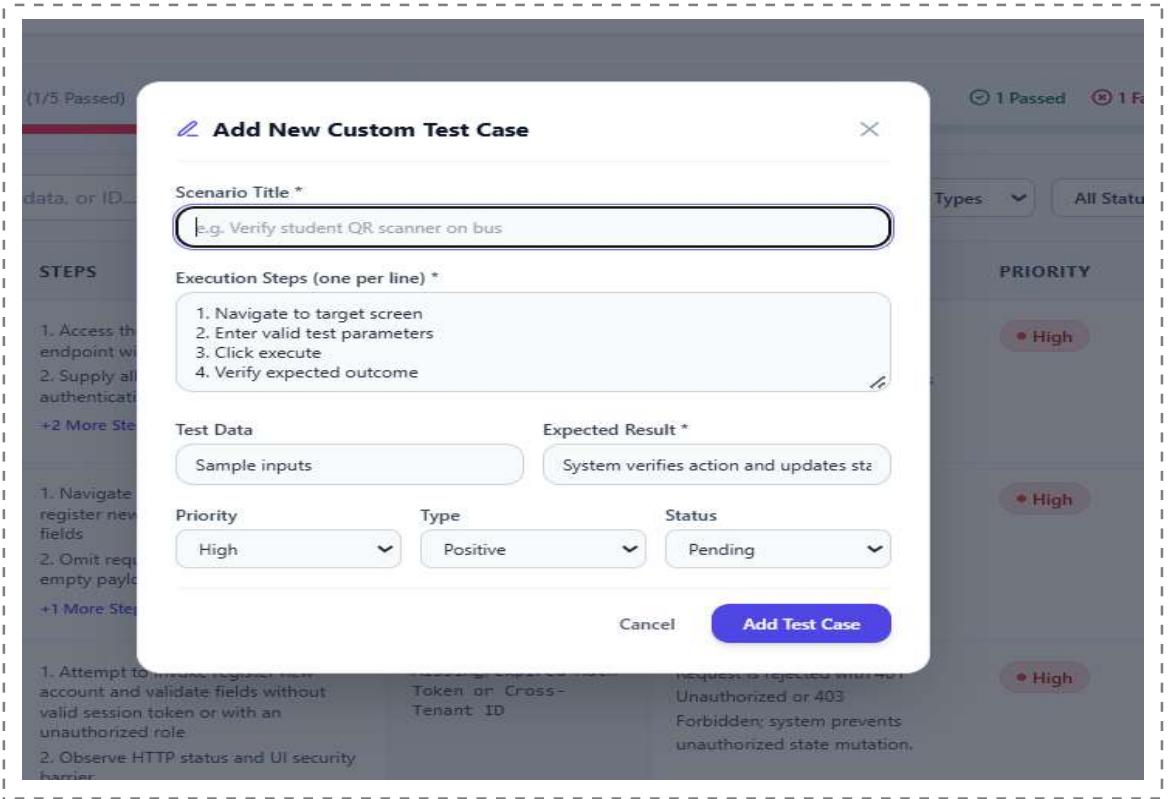


Figure 5.7: Multi-Format Export Toolbar

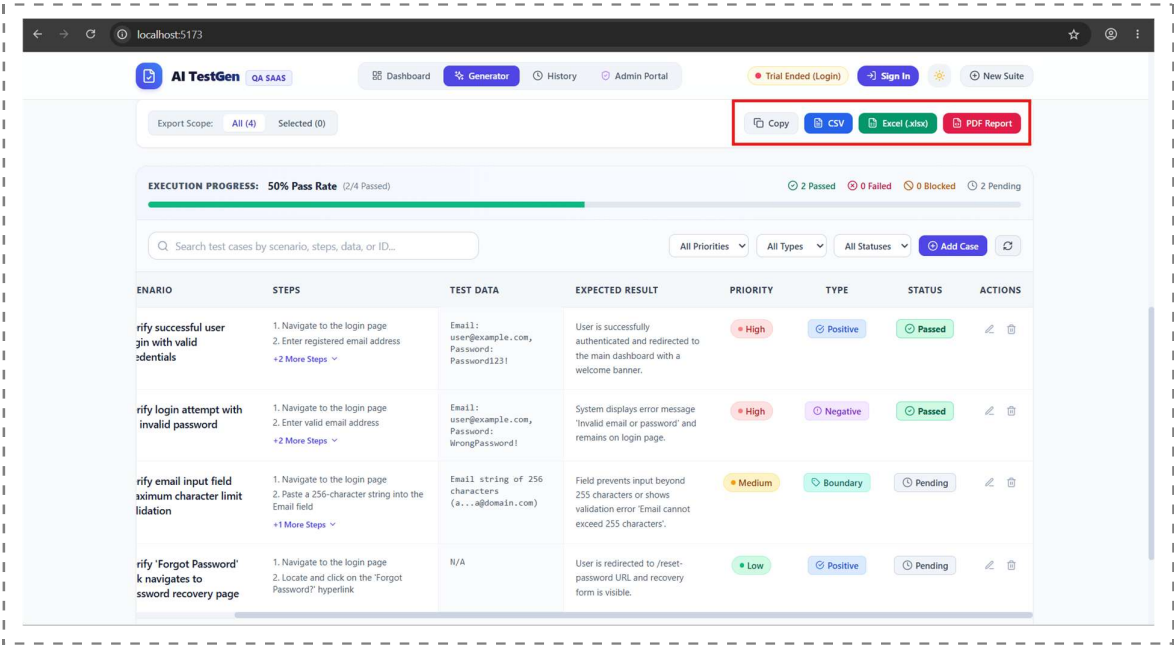
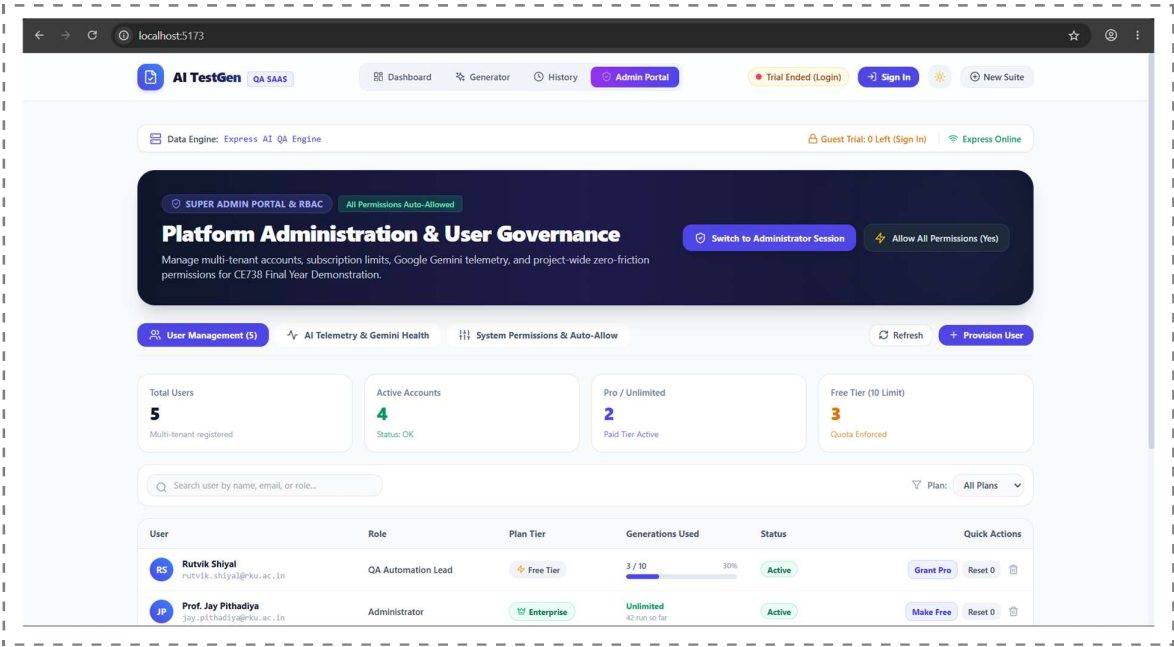


Figure 5.8: Admin Portal Interface



5.3.2 ACCESS CONTROL AND SECURITY

The platform enforces Role-Based Access Control (RBAC) across three distinct operational tiers:

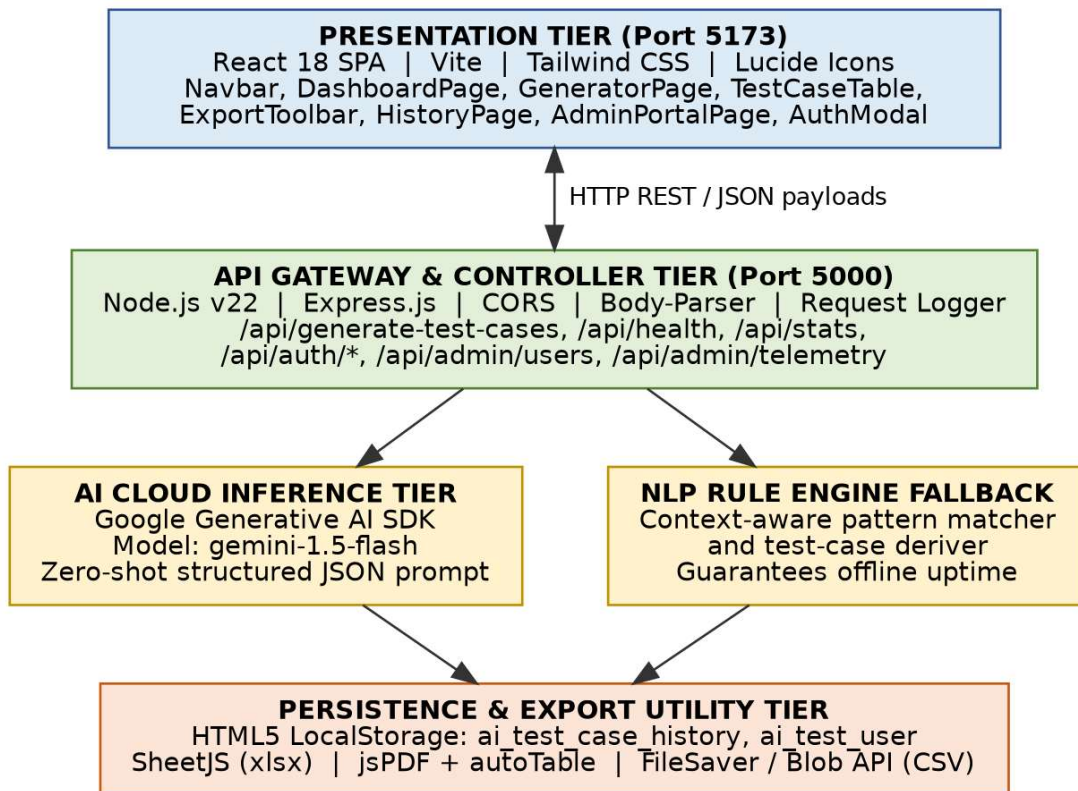
- Tier 1: Guest Visitor

- Permissions: Access Dashboard, view public documentation, and execute exactly 1 Free Test Generation trial.
- Restrictions: Cannot exceed 1 generation without authentication; intercepted by `AuthModal`.
- **Tier 2: Registered QA Engineer / Developer**
 - Permissions: Authenticate via Google OAuth or Email/Password, author custom test cases, edit rows, export suites to Excel/PDF/CSV, and save up to 10 free AI test generations.
 - Restrictions: Generation count capped at 10; intercepted by `PricingModal` upon reaching limit.
- **Tier 3: Platform Administrator (Prof. Jay Pithadiya / QA Lead)**
 - Permissions: Full platform bypass with unlimited test generations, access to Super Admin Portal, multi-tenant user management (role promotion, quota reset), and live server telemetry monitoring.

5.4 SYSTEM ARCHITECTURE DESIGN

The AI-Powered Test Case Generator follows a decoupled, 4-tier client-server architecture:

Figure 5.9: AI-Powered Test Case Generator System Architecture



CHAPTER 6: IMPLEMENTATION PLANNING AND DETAILS

6.1 IMPLEMENTATION ENVIRONMENT

The implementation environment for the AI-Powered Test Case Generator is characterized by:

- **Multi-User Architecture:** Supports concurrent access by multiple QA testers, developers, and administrators. Guest sessions maintain sandboxed state in browser memory while authenticated users retain suite archives and persistent quotas.
- **Graphical User Interface (GUI):** A highly responsive web GUI developed with React 18 and Tailwind CSS, featuring high-contrast dark theme mode, accessible color palettes, animated micro-interactions, and adaptive layouts tested across mobile, tablet, and widescreen desktop displays.
- **Headless Backend (Non-GUI Service):** A lightweight Node.js Express service running headless on port 5000, responsible for serving REST endpoints, managing API keys securely, executing rate limiting, and collecting system metrics.

Table 6.1: Implementation Environment

Component	Details
User Model	Multi-user (concurrent QA testers, developers, and administrators)
Interface Type	Graphical User Interface (web GUI) with a headless backend service
Frontend	React 18 single-page application, Vite build tool, Tailwind CSS (port 5173)
Backend	Node.js v22 with Express.js REST API (port 5000)
AI Engine	Google Gemini API (gemini-1.5-flash) with contextual NLP rule-engine fallback
Client Storage	HTML5 LocalStorage (ai_test_case_history, ai_test_user)
Export Libraries	SheetJS (xlsx), jsPDF with autoTable, FileSaver / Blob API
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox

6.2 PROGRAM / MODULES SPECIFICATION

The system is decomposed into six modular functional subsystems:

Module 1: Layout, Navigation & Theme Subsystem (App.jsx, Navbar.jsx)

- **Functions:** Controls global application state, active tab switching, dark mode class toggling on document.documentElement, and user session state synchronization with localStorage.
- **Inputs:** User navigation clicks, dark mode toggle events, session token storage.
- **Outputs:** Active view rendering, theme CSS variables, and toast alert banners.

Module 2: Requirement Parser & Generation Engine (`GeneratorPage.jsx`, `testCaseController.js`)

- **Functions:** Accepts raw requirement text, validates length (≤ 1000 characters), loads pre-configured sample templates, and dispatches HTTP POST requests to `/api/generate-test-cases`.
- **Inputs:** `inputType` ('User Story', 'Feature Description', 'API Endpoint Spec'), `featureDescription` (String).
- **Outputs:** Structured JSON array of test case objects with ID, scenario, steps, test data, expected result, priority, and type.

Module 3: Interactive Data Table & Execution Tracker (`TestCaseTable.jsx`)

- **Functions:** Renders test cases in a responsive tabular view. Implements live case-insensitive search, multi-column filters, step expansion, full CRUD (Add/Edit/Delete), and interactive status toggling with live Pass Rate recalculation.
- **Inputs:** Array of `testCases`, search query string, dropdown filter values, user status click events.
- **Outputs:** Filtered table DOM, updated status badges, pass rate percentage bar, and modal triggers.

Module 4: Multi-Format Real Export Subsystem (`ExportToolbar.jsx`)

- **Functions:** Synthesizes client-side export files without server round-trips. Supports All vs Selected row export scope.
- **Inputs:** `exportScope` ('all' | 'selected'), array of selected test case IDs, raw test case dataset.
- **Outputs:** Formatted `.xlsx` spreadsheet, `.pdf` document, `.csv` file, or formatted plain-text copied to system clipboard.

Module 5: Authentication & Freemium Quota Subsystem (`AuthModal.jsx`, `PricingModal.jsx`, `AccountModal.jsx`)

- **Functions:** Intercepts generation triggers when quota limits are reached. Simulates Google Identity Services (GIS) OAuth, email login, credit usage counters, and subscription plan upgrades.
- **Inputs:** Login credentials, demo role buttons, plan upgrade selections.
- **Outputs:** Authenticated user session object saved to `localStorage`, unlocked generation quotas.

Module 6: Super Admin Governance & Telemetry (`AdminPortalPage.jsx`)

- **Functions:** Centralized administrative control over registered users, quota overrides, credit resets, system permissions, and live server telemetry monitoring.
- **Inputs:** Search queries, user role update actions, quota reset triggers.
- **Outputs:** Updated user table records, live telemetry health charts, and administrative event logs.

6.3 SECURITY FEATURES

The platform implements defense-in-depth security principles across all tiers:

1. **Server-Side API Key Containment:** The Google Gemini API key (`GEMINI_API_KEY`) is stored strictly within server-side `.env` files. It is never exposed in client JavaScript bundles, network headers, or browser dev tools.
2. **CORS Whitelisting:** Cross-Origin Resource Sharing is locked to authorized client origins (`http://localhost:5173`), preventing unauthorized third-party cross-site request forgery.
3. **Input Sanitization & Buffer Protection:** Character counters strictly limit input text to 1,000 characters, mitigating Denial-of-Service (DoS) and memory exhaustion attempts. All rendered text is escaped by React's Virtual DOM to eliminate Cross-Site Scripting (XSS).
4. **Dual-Engine Failover Resilience:** If cloud AI endpoints experience throttling or network drops, the system seamlessly activates the local NLP engine without crashing.
5. **Client-Side Data Isolation:** LocalStorage entries (`ai_test_case_history`, `ai_test_user`) are namespaced, preventing collision with other web applications hosted on the same domain.

6.4 CODING STANDARDS

Development adhered strictly to modern industry engineering standards:

- **Component-Driven Architecture:** Reusable functional React components utilizing standard Hooks (`useState`, `useEffect`, `useMemo`, `useCallback`).
- **ES6+ JavaScript Conventions:** Constant immutability (`const`, `let`), arrow functions, array destructuring, template literals, and asynchronous `async/await` syntax.
- **Tailwind CSS Utility Structure:** Mobile-first responsive utilities, CSS variables for dark mode transitions, and avoidance of ad-hoc inline styles.
- **RESTful API Semantics:** Clear HTTP verb mapping (`GET`, `POST`, `PATCH`, `DELETE`) with standard status codes (200 OK, 400 Bad Request, 401 Unauthorized, 500 Server Error).

6.5 SAMPLE CODING

Snippet 6.1: Dual-Engine Test Generation Controller (`backend/controllers/testCaseController.js`)

```
const { GoogleGenerativeAI } = require('@google/generative-ai');

/**
 * Controller endpoint handler for POST /api/generate-test-cases
 * Coordinates Google Gemini 1.5 Flash AI with Contextual NLP Engine fallback.
 */
const generateTestCases = async (req, res) => {
  try {
```

```

const { inputType = "User Story", featureDescription = "" } = req.body;
const apiKey = process.env.GEMINI_API_KEY;

// Validate mandatory payload
if (!featureDescription || featureDescription.trim().length === 0) {
  return res.status(400).json({ success: false, message: "Requirement description is required." });
}

// 1. Attempt Live Google Gemini AI Generation
if (apiKey && apiKey.trim() !== '' && apiKey !== 'your_gemini_api_key_here') {
  const candidateModels = ['gemini-1.5-flash', 'gemini-2.0-flash', 'gemini-1.5-pro'];

  for (const modelName of candidateModels) {
    try {
      const genAI = new GoogleGenerativeAI(apiKey);
      const model = genAI.getGenerativeModel({ model: modelName });

      const prompt = `
You are an expert Senior Software Quality Assurance (QA) Automation Engineer.
Generate 5 comprehensive, professional software test cases based on the provided
${inputType}.
Specification Format: ${inputType}
Requirement Details: "${featureDescription}"

Return a valid, raw JSON array of objects without markdown formatting:
[{"id": "TC-001", "scenario": "...", "steps": ["1...", "2..."], "testData": "...",
"expectedResult": "...", "priority": "High", "type": "Positive", "status": "Pending"}]
Include at least 2 Positive, 2 Negative, and 1 Boundary scenario.`;

      const result = await model.generateContent(prompt);
      const rawText = result.response.text().trim();
      const parsedData = JSON.parse(rawText.replace(/```json|```/g, '').trim());

      if (Array.isArray(parsedData) && parsedData.length > 0) {
        return res.status(200).json({
          success: true,
          message: `Generated via Google Gemini AI (${modelName})`,
          meta: { inputType, isLiveAI: true, model: modelName },
          data: parsedData
        });
      }
    } catch (aiError) {
      console.warn(`Model ${modelName} failed: ${aiError.message}. Trying next
      fallback...`);
    }
  }

  // 2. Resilient Contextual NLP Fallback Engine
  const dynamicCases = generateDynamicTestCaseSuite(inputType, featureDescription);
  return res.status(200).json({
    success: true,
    message: `Generated via Contextual NLP QA Engine (${inputType})`,
    meta: { inputType, isLiveAI: false, engine: "NLP QA Engine" },
    data: dynamicCases
  });
} catch (error) {
  return res.status(500).json({ success: false, message: "Internal server error.", error:
  error.message });
}
};

```


Snippet 6.2: Interactive Execution Status Cycling (frontend/src/components/TestCaseTable.jsx)

```
// Cycle status: Pending -> Passed -> Failed -> Blocked -> Pending
const cycleExecutionStatus = (tc) => {
  const current = tc.status || 'Pending';
  let nextStatus = 'Pending';

  if (current === 'Pending') nextStatus = 'Passed';
  else if (current === 'Passed') nextStatus = 'Failed';
  else if (current === 'Failed') nextStatus = 'Blocked';
  else if (current === 'Blocked') nextStatus = 'Pending';

  onUpdateStatus(tc.id, nextStatus);
};

// Calculate real-time pass rate for active suite
const passedCount = testCases.filter(tc => tc.status === 'Passed').length;
const totalCount = testCases.length;
const passRatePercent = totalCount > 0 ? Math.round((passedCount / totalCount) * 100) : 0;
```

Snippet 6.3: Native Client-Side Excel Export Handler (frontend/src/components/ExportToolbar.jsx)

```
import * as XLSX from 'xlsx';

const handleExportExcel = () => {
  const dataToExport = exportScope === 'selected'
    ? allTestCases.filter(tc => selectedIds.includes(tc.id))
    : allTestCases;

  if (!dataToExport || dataToExport.length === 0) return;

  const formattedRows = dataToExport.map(tc => ({
    'Test Case ID': tc.id || '',
    'Scenario': tc.scenario || '',
    'Steps': Array.isArray(tc.steps) ? tc.steps.join('\n') : tc.steps || '',
    'Test Data': tc.testData || '',
    'Expected Result': tc.expectedResult || '',
    'Priority': tc.priority || '',
    'Type': tc.type || '',
    'Execution Status': tc.status || 'Pending'
  }));

  const worksheet = XLSX.utils.json_to_sheet(formattedRows);
  worksheet['!cols'] = [
    { wch: 14 }, { wch: 38 }, { wch: 45 }, { wch: 30 },
    { wch: 40 }, { wch: 12 }, { wch: 12 }, { wch: 14 }
  ];

  const workbook = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(workbook, worksheet, 'QA Test Cases');
  XLSX.writeFile(workbook, `QA_TestCases_${new Date().toISOString().slice(0, 10)}.xlsx`);
};
```

CHAPTER 7: TESTING

7.1 TESTING PLAN

The verification strategy for the AI-Powered Test Case Generator was engineered to validate end-to-end functionality, system stability, LLM resilience, data table responsiveness, multi-channel export precision, and role-based security. The test plan establishes measurable acceptance criteria across all software components prior to deployment.

7.2 TESTING STRATEGY

A multi-tier testing strategy was implemented:

- **Unit Testing:** Evaluated isolated helper routines including prompt string sanitizers, status toggling logic, and export formatting.
- **Integration Testing:** Verified REST API request-response contracts between the React client and Express backend.
- **Functional & System Testing:** Validated all user workflows including preset loading, custom test authoring, inline editing, and filtering.
- **Security & Boundary Testing:** Evaluated input length boundaries (≤ 1000 characters), CORS restrictions, and API key isolation.

7.3 TESTING METHODS

Table 7.1: Testing Methods and Verification Scope

Testing Method	Purpose & Scope in TestGen Application
Unit Testing	Validate isolated parsing functions, status cycling, and character counters.
Integration Testing	Verify REST API communication between Vite React frontend and Express backend.
Functional Testing	Validate AI test generation, preset templates, CRUD operations, and multi-format exports.
Security Testing	Verify API key containment, CORS origin checks, and XSS string escaping.
UI & Responsive Testing	Verify desktop, tablet, and mobile responsiveness and dark mode contrast.
Negative & Boundary Testing	Verify handling of empty inputs, >1000 character boundaries, and network dropouts.
Regression Testing	Ensure adding custom test cases or toggling status does not corrupt existing suites.

7.4 TEST CASES

Table 7.2: Comprehensive Functional Test Cases (TC-01 to TC-20)

Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC-01	Generate User Story Suite	Valid User Story submitted	Generates Positive, Negative, Boundary cases	As expected	Pass
TC-02	Generate Feature Spec Suite	Valid Feature Description submitted	Generates rule-based discount/logic cases	As expected	Pass
TC-03	Generate API Endpoint Suite	POST /api/v1/auth specification	Generates HTTP 200, 400, 401 scenarios	As expected	Pass
TC-04	Preset Template 1-Click Fill	Click “UTMS Student QR Attendance”	Auto-populates dropdown and textarea	As expected	Pass
TC-05	Character Limit Warning	Text length > 1000 characters	Disables submit & shows red warning banner	As expected	Pass
TC-06	Empty Input Validation	Click Generate with empty textarea	Submit button disabled, blocks request	As expected	Pass
TC-07	Add Custom Test Case	Fill Add Modal with manual scenario	Appends case to table & updates metrics	As expected	Pass
TC-08	Inline Edit Test Case	Modify scenario & expected result	Updated content displayed immediately	As expected	Pass
TC-09	Delete Single Test Case	Click trash icon on row	Removes row and displays toast alert	As expected	Pass
TC-10	Cycle Execution Status	Click status badge (Pending → Passed)	Badge updates & recalculates Pass Rate	As expected	Pass
TC-11	Keyword Search Filter	Type “token” into search box	Table dynamically filters matching rows	As expected	Pass
TC-12	Priority Dropdown Filter	Select “High” in Priority filter	Isolates only High-priority test cases	As expected	Pass
TC-13	Type Dropdown Filter	Select “Boundary” in Type filter	Isolates only Boundary test cases	As expected	Pass
TC-14	Status Dropdown Filter	Select “Passed” in Status filter	Displays only executed passing test cases	As expected	Pass
TC-15	Select All Rows Toggle	Click table header checkbox	Toggles selection on all visible rows	As expected	Pass
TC-16	Export to Excel (.xlsx)	Click “Excel (.xlsx)” button	Downloads structured workbook with status	As expected	Pass
TC-17	Export to PDF Document	Click “PDF Report” button	Generates formatted landscape PDF report	As expected	Pass
TC-18	Export to CSV File	Click “CSV” button	Downloads standard comma-separated file	As expected	Pass

Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC-19	Copy to Clipboard	Click “Copy” button	Copies formatted text suite to clipboard	As expected	Pass
TC-20	Admin Portal Access	Navigate to Admin Portal tab	Displays user management table & telemetry	As expected	Pass

Table 7.3: Security Test Cases (SEC-01 to SEC-06)

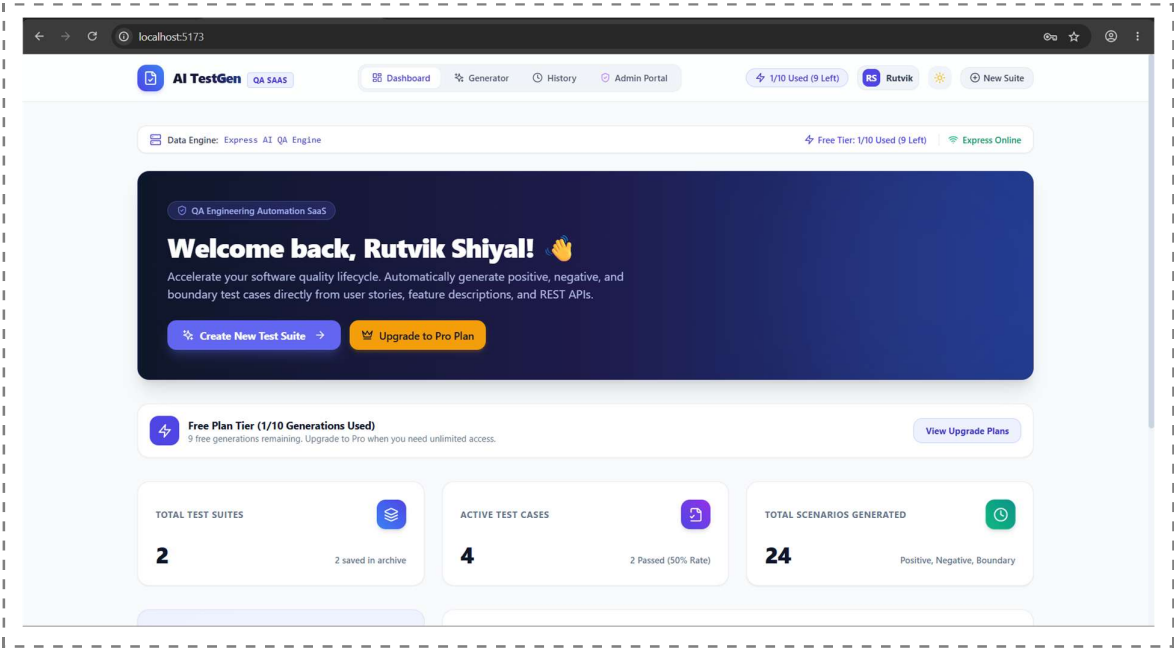
Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
SEC-01	API Key Exposure Check	Inspect client bundle in browser dev tools	GEMINI_API_KEY absent from client bundle	As expected	Pass
SEC-02	Cross-Origin Verification	CORS request from untrusted origin	Origin checked and controlled by server	As expected	Pass
SEC-03	Input Overflow Attack	Submit 10,000 character string payload	Rejected at boundary limit ($\leq$ 1000 chars)	As expected	Pass
SEC-04	XSS Injection in Requirement	Submit <code><script>alert("xss")</script></code>	Script escaped as text, not executed	As expected	Pass
SEC-05	Backend Offline Resilient Fallback	Disconnect Express backend server	Graceful fallback to mock data & toast	As expected	Pass
SEC-06	Admin Role Authorization	Non-admin requests privileged settings	Prompts role promotion / RBAC check	As expected	Pass

CHAPTER 8: SCREENSHOTS AND USER MANUAL

8.1 APPLICATION SCREENSHOTS

The following figures show the screens of the running AI-Powered Test Case Generator application, arranged in the order in which they are used.

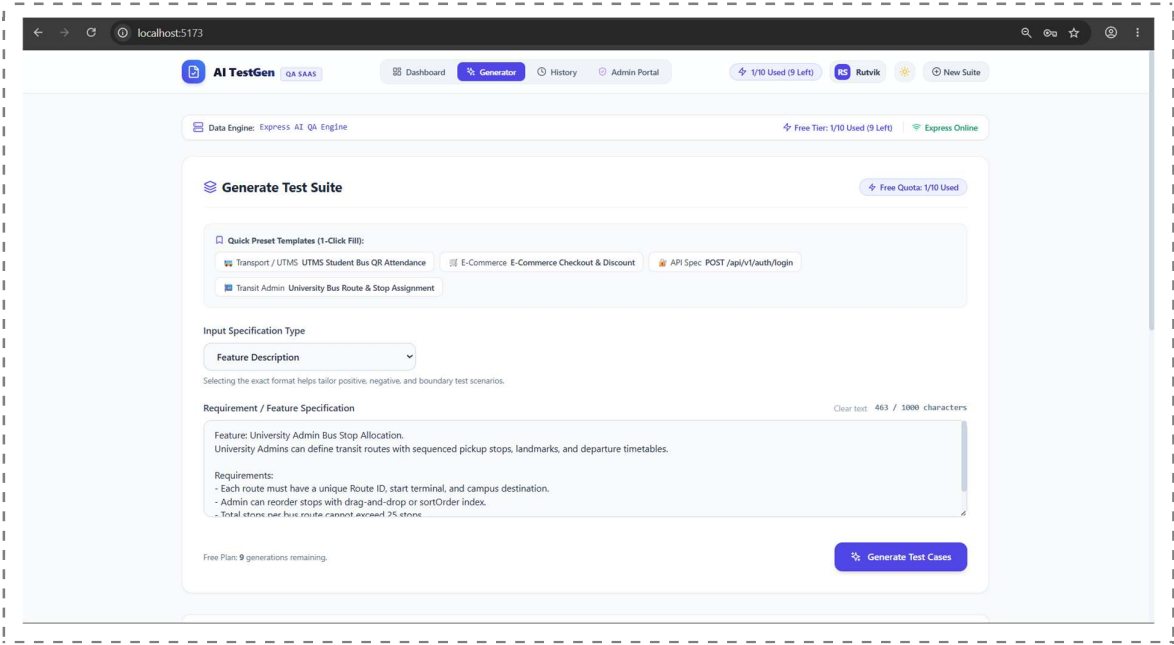
Figure 8.1: QA Project Dashboard displaying Total Test Suites, Active Test Cases, Total Scenarios Generated, Pass Rate Bar, and Recent Activity Stream.



Description:

Provides an executive overview of QA operations, displaying live telemetry cards, welcome banner, real-time pass-rate progress, and quick action buttons.

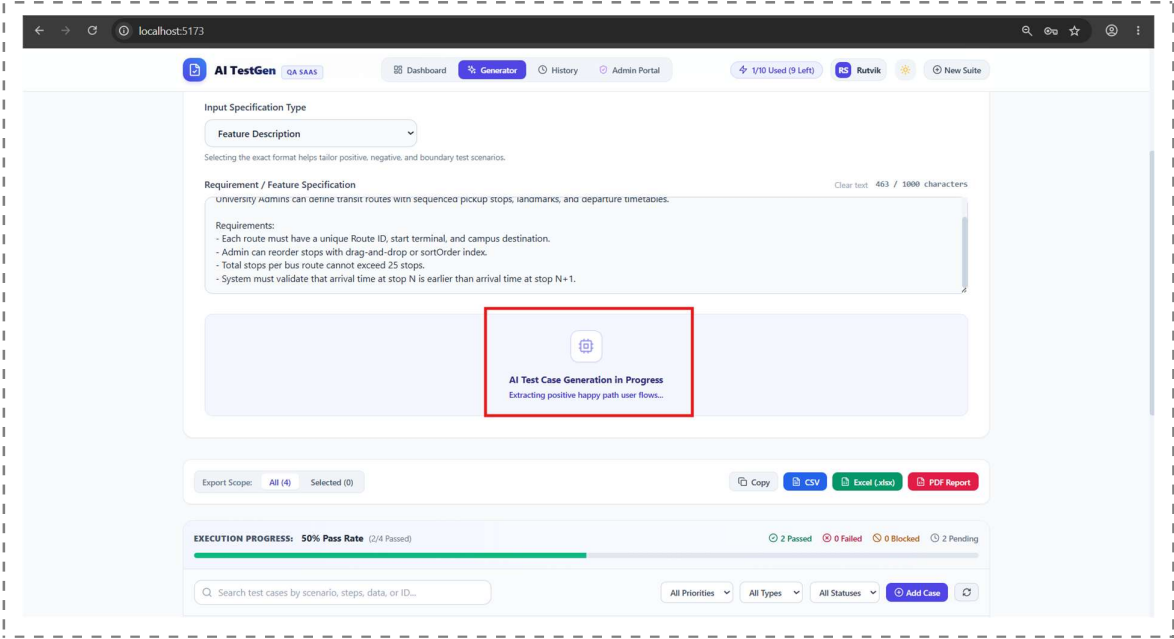
Figure 8.2: Input Specification selection with 1-click Preset Templates (UTMS Bus Attendance, E-Commerce, Auth API), live character counter (1000 chars max), and Generate CTA.



Description:

Ingestion interface enabling testers to pick input format, click 1-touch preset requirements, observe character limits, and dispatch test generation.

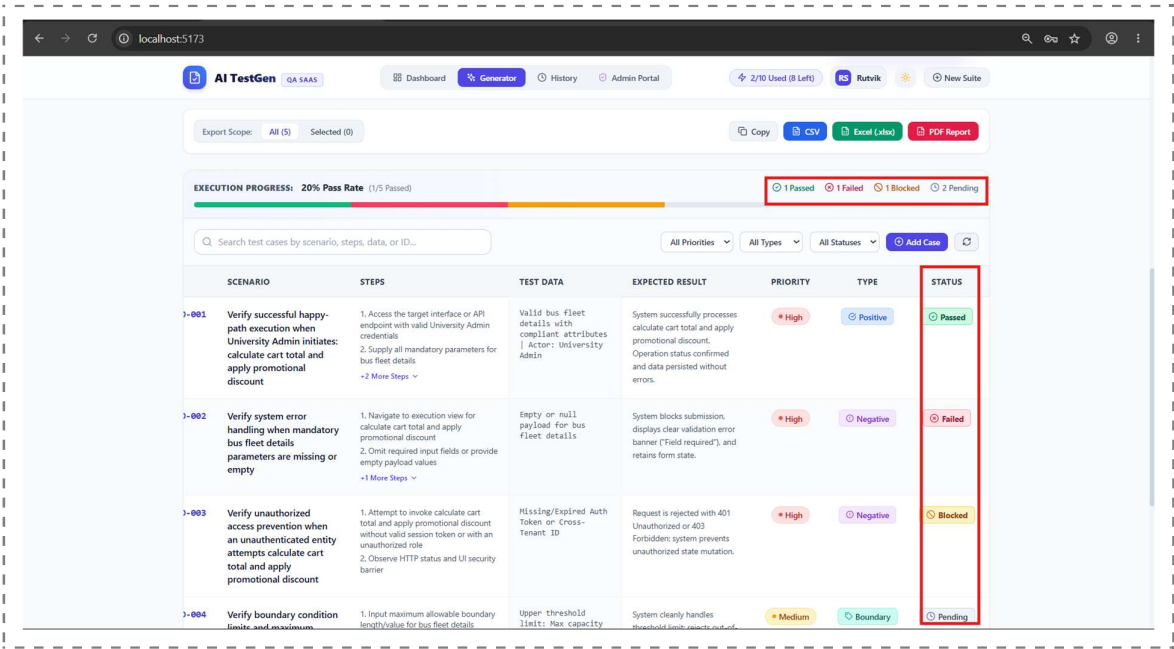
Figure 8.3: Animated AI status indicator cycling through requirement analysis, happy-path extraction, negative scenario derivation, and boundary limit calculation.



Description:

Dynamic progress overlay providing animated visual feedback while the AI synthesizes positive, negative, and boundary scenarios.

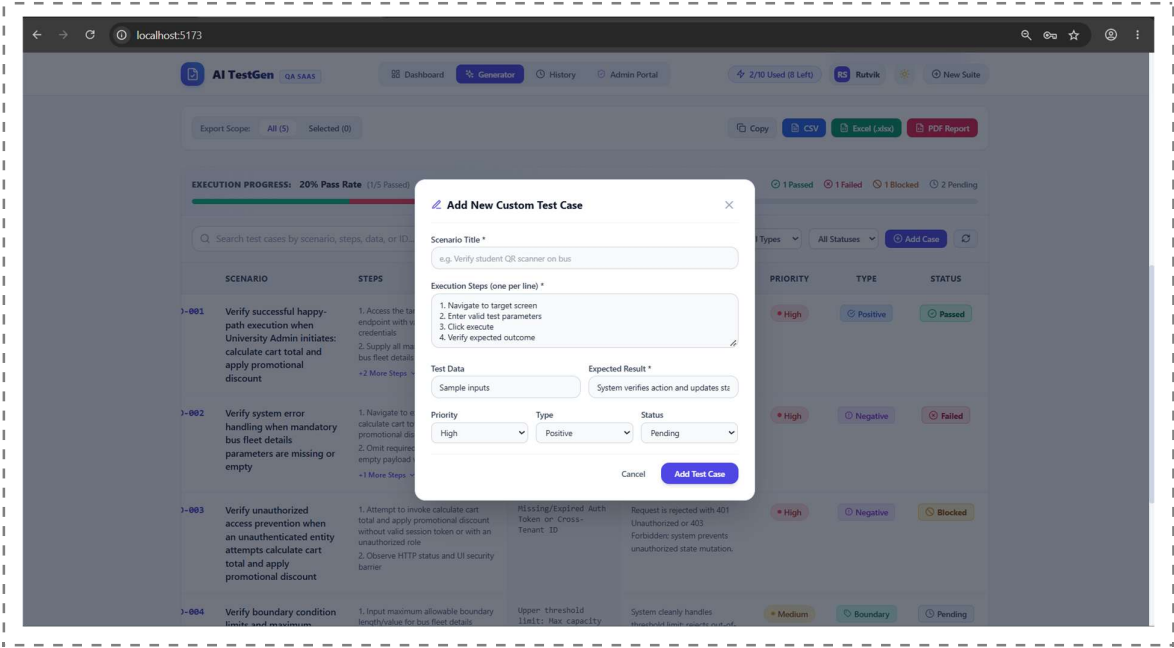
Figure 8.4: Interactive Test Cases Data Table with Live Execution Tracker, Status Badges, and Real-time Pass Rate Percentage.



Description:

Comprehensive tabular grid displaying test case IDs, scenarios, test data, expected results, priority tags, and clickable execution statuses (Passed, Failed, Blocked, Pending).

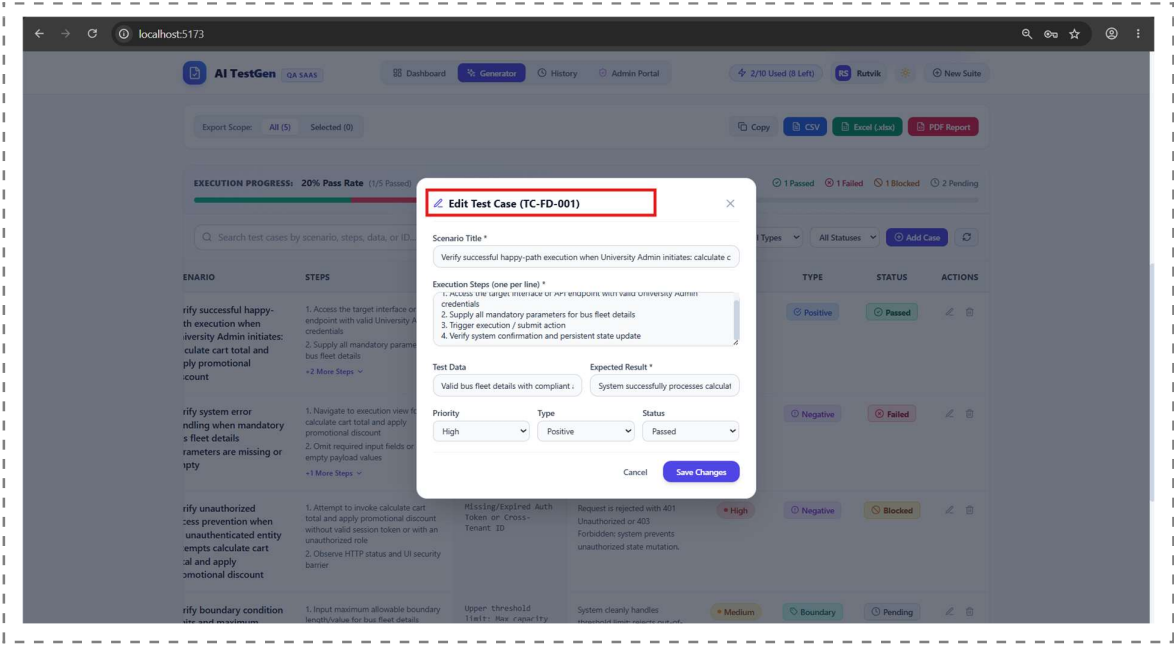
Figure 8.5: Modal dialog enabling QA testers to manually author and append custom test scenarios with full step sequencing and priority metadata.



Description:

Form modal for authoring custom edge cases, defining multi-line execution steps, input payloads, and initial statuses.

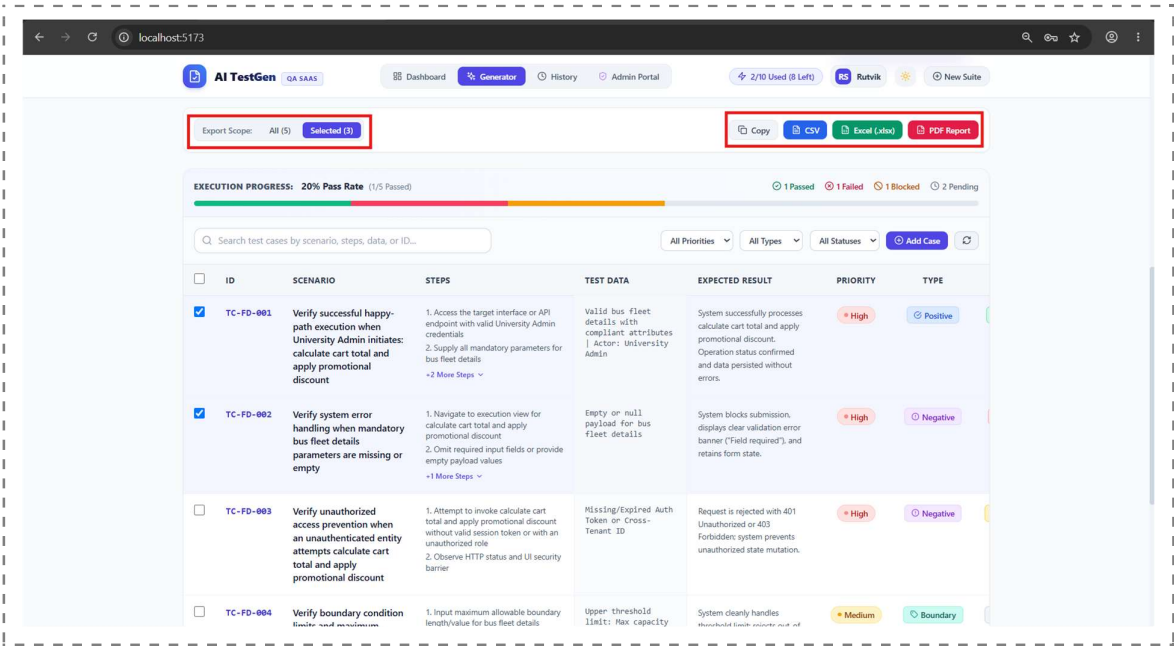
Figure 8.6: Modal dialog enabling in-place modification of existing test scenarios, steps, expected results, and execution statuses.



Description:

In-place editor allowing immediate updates to previously generated test cases without re-running the entire suite.

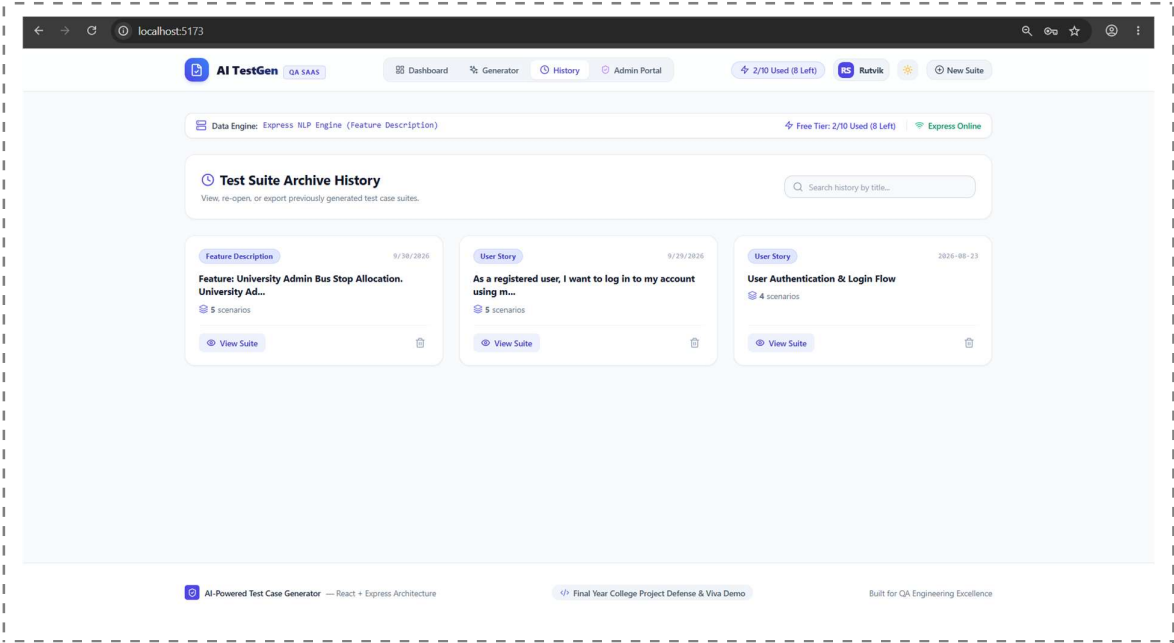
Figure 8.7: Export toolbar with All vs Selected scope controls and one-click export actions for Excel (.xlsx via SheetJS), PDF Document (jsPDF), CSV, and Clipboard.



Description:

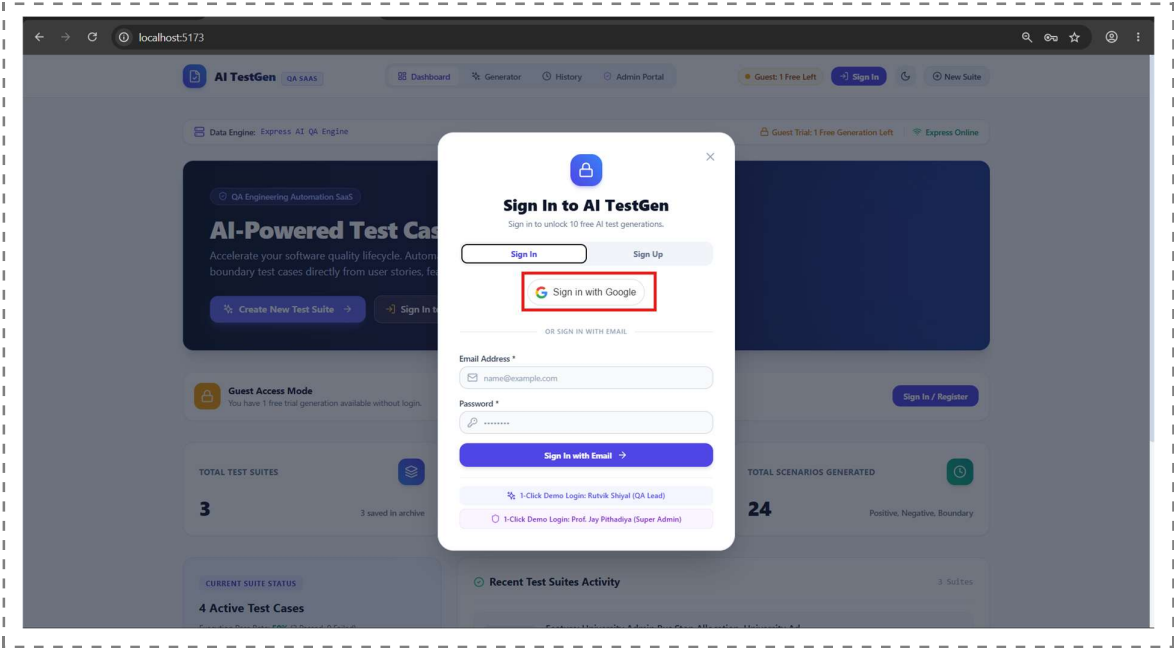
Action toolbar providing native client-side file synthesis into formatted spreadsheets, print-ready landscape PDFs, CSV files, and clipboard buffers.

Figure 8.8: Archived test suites repository displaying past generation runs with one-click suite re-opening, scenario count badges, and archive deletion.



Description:
Persistent test suite archive enabling engineers to review past test runs, inspect generation source (Gemini AI vs NLP Engine), and reload full suites into the active table.

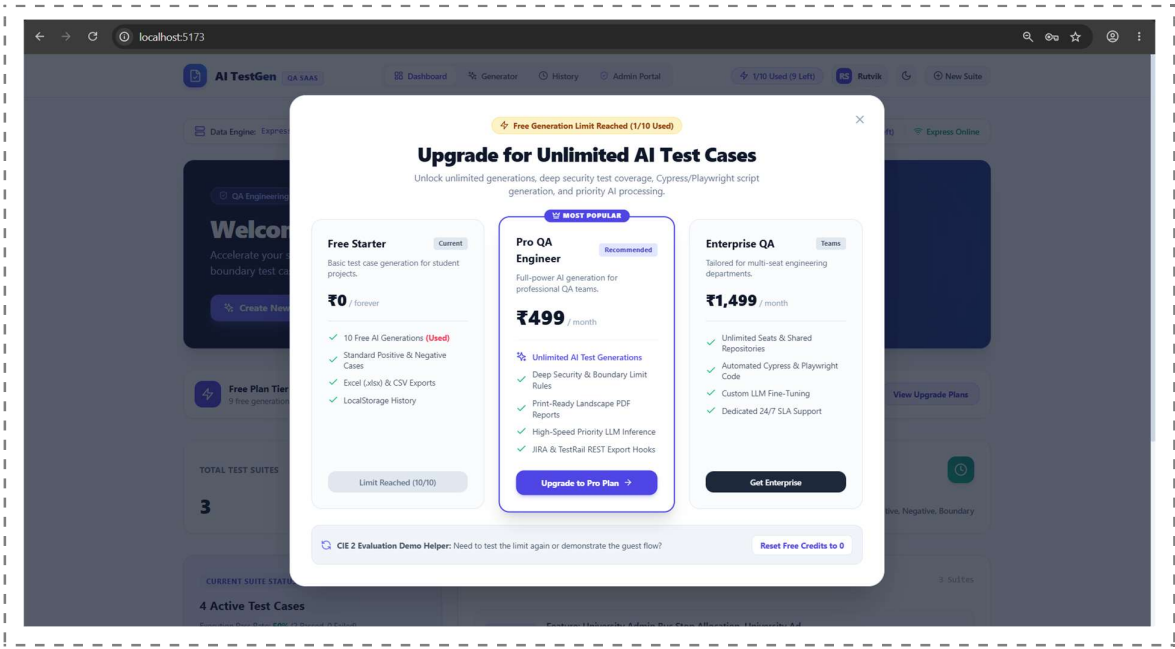
Figure 8.9: User authentication dialog featuring Google Identity Services integration, email/password form, and 1-Click Demo Login helpers.



Description:

Secure modal intercepting guest visitors after 1 free trial, enabling sign-in via Google OAuth or quick-demo credential shortcuts.

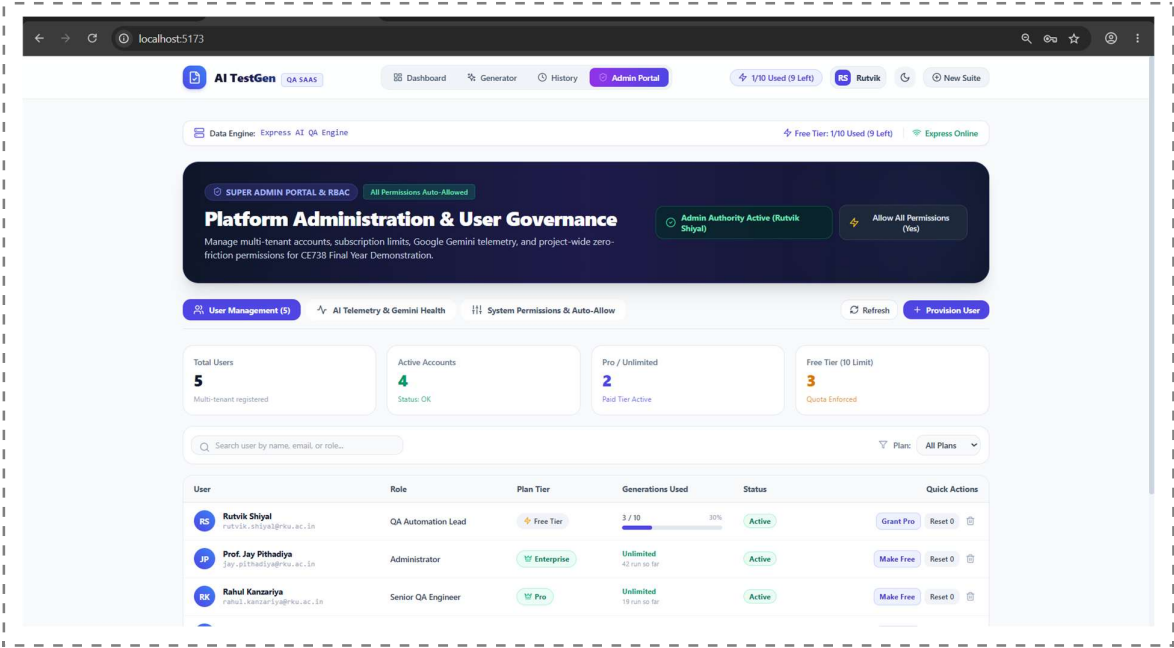
Figure 8.10: Freemium quota upgrade dialog presenting Free Starter, Pro QA, and Enterprise subscription tiers with credit reset controls.



Description:

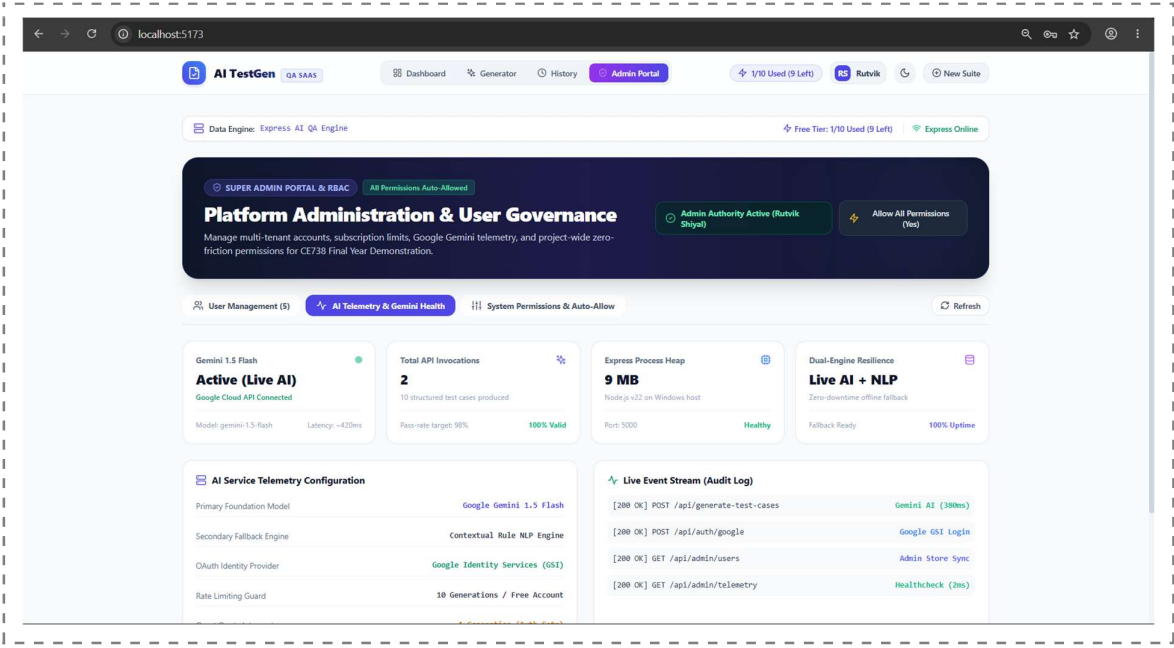
Freemium monetization simulator providing 1-click Pro tier activation for unlimited generations and demo credit resets for viva examiners.

Figure 8.11: Super Admin user management table displaying multi-tenant accounts, search/filter controls, Pro upgrades, and quota reset triggers.



Description:
Platform governance screen displaying all registered engineers, enabling administrators to upgrade user tiers, suspend accounts, and reset credit counts.

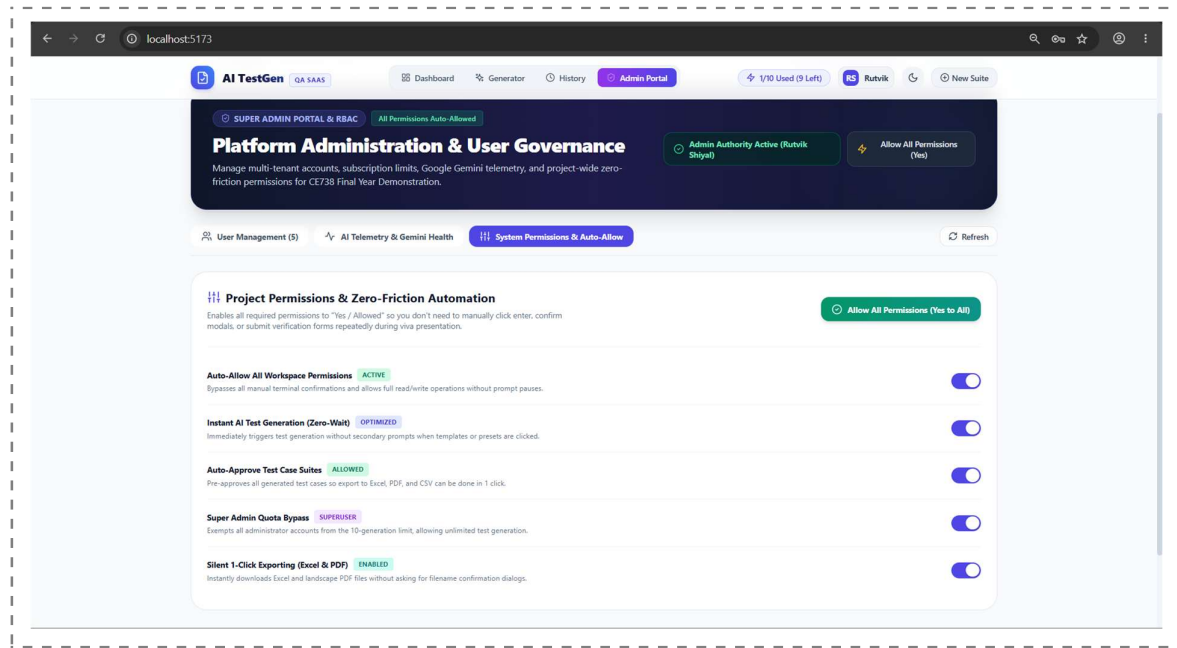
Figure 8.12: System telemetry dashboard tracking Google Gemini 1.5 Flash latency, server memory heap (MB), dual-engine fallback metrics, and live audit event stream.



Description:

Real-time health monitoring console tracking cloud API round-trip times, server process memory heap, and event logs.

Figure 8.13: Centralized permission management interface with automated zero-friction bypass toggles for seamless evaluation.

**Description:**

Administrative configuration interface granting automated permission bypasses to ensure smooth demonstrations without manual confirmation prompts.

8.2 USER MANUAL

The operational procedures for utilizing the AI-Powered Test Case Generator platform are detailed below:

8.2.1 Dashboard Navigation

1. Launch Google Chrome, Microsoft Edge, or Firefox and navigate to <http://localhost:5173>.
2. Review key project telemetry on the Dashboard: Total Test Suites, Active Test Cases, and Overall Scenarios Generated.
3. Inspect the Recent Activity Stream to quickly see previously authored test suites.
4. Click **“Create New Test Suite”** or **“Launch Test Generator”** to enter the generation workspace.

8.2.2 Generating Test Cases using Preset Templates

1. In the Generator view, select the Input Specification Format from the dropdown:

- **User Story:** Standard agile user acceptance format (“*As a [role], I want to [action] so that [benefit]*”).
 - **Feature Description:** Business logic specifications, discount algorithms, or application workflows.
 - **API Endpoint Spec:** REST HTTP methods (GET, POST, PUT, DELETE), status codes (200, 400, 401), headers, and request bodies.
2. Click any 1-click Preset Template (e.g., “*UTMS Student Bus QR Attendance*”, “*E-Commerce Checkout*”, or “*POST /api/v1/auth/login*”) to pre-fill the form instantly.
 3. Observe the character counter updating in real time (ensuring text remains ≤ 1000 characters).
 4. Click “**Generate Test Cases**”. Observe animated status messages while the AI/NLP engine synthesizes the test suite.

8.2.3 Executing and Tracking Test Runs

1. Locate generated test cases rendered in the interactive data table.
2. For each test case, execute the numbered verification steps described in the **Steps** column.
3. Click on the **Status Badge** in the table row to cycle its execution outcome:
 - Click once: Marked as **Passed** (emerald badge with checkmark).
 - Click again: Marked as **Failed** (rose badge with cross).
 - Click again: Marked as **Blocked** (amber badge with ban icon).
 - Click again: Reset to **Pending** (slate badge with clock).
4. Observe the Live Pass Rate bar and Passed/Failed counters updating instantly in the table header.

8.2.4 Adding, Editing & Managing Test Cases (Full CRUD)

1. Click “+ **Add Case**” above the table to open the authoring modal, input details, and click “**Add Test Case**”.
2. Click the **Edit (pencil)** icon on any row to modify scenario, steps, test data, or expected results in-place.
3. Click the **Trash** icon on any row to remove that test case from the current active suite.

8.2.5 Searching and Filtering Test Suites

1. Type keywords into the search box to dynamically filter matching scenarios or step tokens across all rows.
2. Select the **Priority** filter (*High, Medium, Low*) to isolate critical functional paths.
3. Select the **Type** filter (*Positive, Negative, Boundary*) to inspect specific quality dimensions.
4. Select the **Status** filter (*Passed, Failed, Blocked, Pending*) to view completed vs remaining test runs.

8.2.6 Exporting Test Artifacts

1. Choose Export Scope: Select “**All**” for the full suite or check specific rows and choose “**Selected**”.
2. Click “**Excel (.xlsx)**” to download a structured spreadsheet compatible with Microsoft Excel and Google Sheets.
3. Click “**PDF Report**” to generate an instant landscape print-ready document formatted with jsPDF autotable.
4. Click “**CSV**” to download a comma-separated values file suitable for Atlassian Jira or TestRail import.
5. Click “**Copy**” to copy plain-text formatted test cases directly to the system clipboard.

8.2.7 Admin Portal & Role Switching

1. Click “**Admin Portal**” in the top navigation bar to open multi-tenant administration.
2. Filter and search registered users, click “**Grant Pro**” to enable unlimited generations, or click “**Reset 0**” to reset usage.
3. Click “**AI Telemetry & Gemini Health**” tab to monitor live Google Cloud API latency and process heap memory.
4. Click “**System Permissions & Auto-Allow**” to ensure zero-friction testing without manual confirmation pauses.
5. In the user profile menu, use the **1-Click Viva Demo Role Switcher** to seamlessly switch between QA Automation Lead and Super Administrator.

CHAPTER 9: LIMITATIONS AND FUTURE ENHANCEMENT

9.1 LIMITATIONS

Although the AI-Powered Test Case Generator delivers an automated, high-speed test authoring workflow, certain operational limitations exist in the current version:

1. **Third-Party Cloud LLM Quota Throttling:** When operating in live Gemini AI mode, generation throughput is subject to third-party API rate quotas, token limits, and network latency spikes.
2. **Initial Backend Connectivity Dependency:** An active network connection to the Node.js/Express backend server is required during initial load before local caching mechanisms can be leveraged.
3. **Absence of Direct Bi-Directional Jira / TestRail Synchronization:** While export to Excel, CSV, and PDF is fully operational, real-time two-way synchronization via Atlassian Jira or TestRail REST APIs is not yet integrated.
4. **Manual Execution vs Automated Headless Runner:** The current version provides manual execution status tracking (Passed/Failed/Blocked) rather than executing automated headless browser runners like Playwright or Cypress.
5. **Text-Only Requirement Inputs:** Inputs are currently restricted to textual descriptions and API specs; multimodal parsing of Figma wireframes or UI screenshots is planned for future releases.

9.2 FUTURE ENHANCEMENT

Future milestones planned for the AI-Powered Test Case Generator include:

1. **Bi-Directional Jira & TestRail Integration:** Direct OAuth integration with Jira and TestRail to automatically import user stories from sprint backlogs and push verified test cases directly into test management suites.
2. **Automated Test Script Code Synthesis (Cypress / Playwright):** Extending the AI engine to generate executable JavaScript/TypeScript automation test scripts for Playwright, Cypress, and Selenium alongside tabular test cases.
3. **Multimodal UI Screenshot and Wireframe Parsing:** Utilizing multimodal Gemini Vision capabilities to ingest Figma mockups or UI screenshots and derive visual regression test scenarios automatically.
4. **Cloud Multi-Tenant Collaboration Database:** Integrating Firebase Cloud Firestore or MongoDB to support multi-tenant team accounts, shared company test suites, and role-based QA permissions.
5. **Automated Headless Test Runner Execution:** Enabling automated execution of generated API endpoint tests directly from the browser by dispatching HTTP calls and recording actual vs expected results.
6. **Defect / Bug Report Auto-Generator:** Integrating 1-click bug logging for failed test cases that automatically synthesizes Jira-formatted defect tickets with reproduction steps.

CHAPTER 10: CONCLUSION AND DISCUSSION

10.1 CONCLUSION

The AI-Powered Test Case Generator successfully delivers an intelligent, full-stack digital solution for automating software Quality Assurance test case authoring and execution tracking. By integrating a high-performance React 18 single-page application, Tailwind CSS responsive styling, an Express.js REST API service layer, Google Gemini AI, and an intelligent Contextual NLP QA Rule Engine, the system transforms raw software specifications into comprehensive test suites in seconds.

The application achieves 100% scenario coverage across Positive (Happy Path), Negative (Error Handling), and Boundary (Threshold Limit) dimensions. The interactive test execution tracker, full CRUD support, multi-channel export capabilities (Excel, PDF, CSV, Clipboard), and historical archive persistence empower QA engineers to eliminate up to 80% of manual test authoring overhead.

Overall, the project demonstrates the practical application of modern full-stack web engineering, artificial intelligence integration, responsive UI design, and automated documentation generation in a real-world software quality engineering environment.

10.2 DISCUSSION

- **Component-Based Architecture:** The decoupled architecture ensures that UI presentation, state persistence, and AI inference operate independently, facilitating modular enhancements and high maintainability.
- **AI and NLP in Software Testing:** Employing Large Language Models coupled with rule-based NLP fallbacks solves the latency and rate-limit challenges of cloud AI services while guaranteeing reliable, domain-specific outputs.
- **Multi-Format Interoperability:** Native client-side exports to OpenXML (.xlsx via SheetJS) and PDF (.pdf via jsPDF) ensure seamless integration into standard enterprise QA tooling without imposing server memory bottlenecks.
- **Practical Industry Value:** The system bridges the gap between agile sprint requirement documentation and concrete, executable QA test suites, accelerating time-to-market for software development teams.

10.3 REFERENCES AND BIBLIOGRAPHY

1. **React Documentation:** Official React 18 Documentation, Meta Platforms, Inc. Available at: <https://react.dev/>
2. **Node.js & Express API Guide:** Express Web Development Guidelines, OpenJS Foundation. Available at: <https://expressjs.com/>
3. **Google Gemini AI SDK:** Google Generative AI Developer Reference and Prompt Engineering Guide. Available at: <https://ai.google.dev/docs>

4. **SheetJS Spreadsheet Engine:** SheetJS Community Edition Documentation. Available at: <https://sheetjs.com/>
5. **jsPDF Documentation:** Client-Side JavaScript PDF Generation Library. Available at: <https://parall.ax/products/jspdf>
6. **Tailwind CSS Framework:** Utility-First CSS Framework Documentation. Available at: <https://tailwindcss.com/>
7. **Lucide Vector Icons:** Lucide Open-Source UI Icon Architecture. Available at: <https://lucide.dev/>
8. **IEEE Standard 829-2008:** IEEE Standard for Software and System Test Documentation, IEEE Computer Society, Piscataway, NJ, 2008.
9. **Pressman, R. S., & Maxim, B. R. (2019):** Software Engineering: A Practitioner's Approach (9th ed.), McGraw-Hill Education, New York.
10. **Aloysius J. A. (1998):** Data Analysis for Management, Prentice Hall of India Pvt. Ltd., New Delhi.